

Resolución: Contar Unos en un Vector Binario Ordenado

Este documento presenta el diseño de un algoritmo de **Divide y Vencerás** para contar el número de unos en un vector binario ($a[i] \in \{0, 1\}$) ordenado de tamaño n en tiempo logarítmico $O(\log n)$.

a) Idea del Algoritmo

Dado que el vector a está ordenado y solo contiene ceros y unos, siempre tendrá la siguiente forma:

$$a = [0, 0, \dots, 0, 1, 1, \dots, 1]$$

El problema de contar cuántos unos hay en el vector se reduce a **encontrar el índice de la primera aparición del número 1**.

Si logramos encontrar el primer índice p tal que $a[p] = 1$ (utilizando índices con base 1):

- Todos los elementos desde la posición p hasta el final n serán obligatoriamente unos.
- La cantidad total de unos será simplemente:

$$\text{Cantidad de unos} = n - p + 1$$

- Si no existe ningún 1 en el vector, el resultado será 0.

Para encontrar esta primera transición de 0 a 1 en tiempo $O(\log n)$, aplicamos una variante de la **Búsqueda Binaria** (Divide y Vencerás).

b) Criterios de Descarte

Al evaluar la posición media mid del rango actual de búsqueda $[izq, der]$:

1. Si $a[mid] = 1$: El primer uno de la secuencia puede ser el propio mid o puede estar más a la izquierda.
 - **Decisión:** Comprobamos si es el primer elemento del vector ($mid = 1$) o si el elemento anterior es un cero ($a[mid - 1] = 0$). Si se cumple alguna de estas, hemos encontrado el primer uno y devolvemos mid . En caso contrario, descartamos la mitad derecha y seguimos buscando a la izquierda en $[izq, mid - 1]$.
2. Si $a[mid] = 0$: Dado que el vector está ordenado, todos los elementos a la izquierda de mid también son obligatoriamente ceros. El primer uno tiene que estar estrictamente en la mitad derecha.
 - **Decisión:** Descartamos la mitad izquierda y seguimos buscando en $[mid + 1, der]$.

c) Algoritmo en Pseudocódigo

A continuación se detalla el pseudocódigo recursivo que busca el índice del primer uno y calcula el total utilizando índices base 1:

```
Funcion BuscarPrimerUno(a, izq, der)
Inicio
    // Caso Base: El rango se ha cruzado y no se ha encontrado ningún 1
    Si izq > der Entonces
        Retornar -1
    FinSi

    // Calcular el punto medio
    mid <- ParteEntera((izq + der) / 2)

    Si a[mid] = 1 Entonces
        // Comprobar si es el primer 1 del vector completo
        Si mid = 1 o a[mid - 1] = 0 Entonces
            Retornar mid
        Sino
            // Seguir buscando el origen de los unos a la izquierda
            Retornar BuscarPrimerUno(a, izq, mid - 1)
        FinSi
    Sino
        // Es un 0, el primer uno tiene que estar a la derecha
        Retornar BuscarPrimerUno(a, mid + 1, der)
    FinSi
FinFuncion

Funcion ContarUnos(a, n)
Inicio
    primer_uno <- BuscarPrimerUno(a, 1, n)

    Si primer_uno = -1 Entonces
        Retornar 0 // No hay unos en el vector
    Sino
        Retornar n - primer_uno + 1
    FinSi
FinFuncion
```

d) Análisis de Coste

- **Complejidad Temporal:** $O(\log n)$. En cada paso de la recursión dividimos el espacio de búsqueda exactamente por la mitad, realizando comparaciones elementales de tiempo constante $O(1)$.
- **Complejidad Espacial:** $O(\log n)$ debido al uso de la memoria en la pila de llamadas de la recursión, la cual tiene una profundidad máxima equivalente a la altura del árbol de división recursiva ($\log_2 n$).